

Writing Architecture Decision Records (ADRs)

A student guide for deciding **when** to write an ADR, **how to name and structure** it, and **how to keep it linked** to features and Cursor rules.

Index & template: [README.md](#)

Examples in this repo: [ADR-0001](#) (client-server), [ADR-0002](#) (security), [ADR-0003](#) (MySQL)

Related: [features/framework.md](#) · [writing feature requirements](#) · [writing feature design](#)

What an ADR is

An **Architecture Decision Record** is a short, durable document that records a **cross-cutting choice** and the **reasons** behind it: context, decision, consequences, and alternatives you rejected.

Artifact	Question it answers
Feature spec (features/feature-N-*.md)	<i>What</i> must the product do in this slice?
Cursor rule (.cursor/rules/*.mdc)	<i>How</i> must we implement day to day?
ADR (docs/adr/NNNN-....md)	<i>Why</i> did we choose this approach for the whole app (or a major subsystem)?
NFR table (docs/nfr/)	What quality bars apply app-wide?
Living reference	What API/schema/rules exist on dev <i>now</i> ?

ADRs outlive a single feature. Feature 2 and Feature 5 both rely on ADR-0001's "server is source of truth" and ADR-0002's ownership/ 404 rules — without re-arguing them in every spec.

When to write an ADR (vs a feature or a rule)

Situation	Write...
New user-facing behavior for one feature	Feature spec (stories, FRs, AC)
Ongoing coding pattern (API shape, Vue style, test layout)	Cursor rule
Significant stack or architecture choice	ADR
Security or data-isolation model	ADR
Database engine / persistence approach	ADR
Deviation from an existing rule or ADR	ADR — then update the rule/spec
Button label or empty-state copy on one screen	Screen Requirements in the feature (not an ADR)

Rule of thumb: if the choice will still matter after three features, and teammates might otherwise re-decide it, write an ADR.

Write the ADR before or with the first feature that depends on it. Link it from that feature's header (****Related:****).

Examples from this project

ADR	Why it is an ADR (not a story)
-----	--------------------------------

0001 Client-server multi-user	Affects every feature: SPA vs server truth, /todo/, JWT+session, monorepo
0002 Layered security	Trust boundary, 404 vs 403, bcrypt, "API enforces / UI is UX only"
0003 MySQL relational DB	Engine + Sequelize + separate test DB — not "add lists table" (that is Feature 2)

Not ADRs: "User can create a list" (Feature 2 story), "use `oc-cta` on primary buttons" (ui-style rule), "p95 < 200 ms" (NFR row).

Naming and filing

```
docs/adr/
  README.md           ← index + short template
  writing-adrs.md      ← this guide
  NNNN-short-kebab-title.md ← one decision per file
```

Part	Principle	Example
Number	Four digits, sequential; never reuse	0001, 0002, 0003
File slug	kebab-case, specific capability	0003-mysql-relational-database.md
Title (# ADR-NNNN: ...)	Short decision name, not a novel	MySQL relational database
Status	Proposed → Accepted (Or Deprecated / Superseded by ADR-XXXX)	See header

Naming principles

1. **Name the decision, not the whole product.** Good: Layered security architecture . Weak: How the Todo app works .
2. **Be specific enough to find later.** Prefer MySQL relational database over Database .
3. **Match file slug to title.** Same words, kebab-case in the filename.
4. **One decision per file.** Do not combine "use MySQL" and "use bcrypt" unless they are inseparable; security got its own ADR-0002.
5. **Update the [README index](#)** when you add or supersede an ADR.

Structure (required sections)

Copy from [README.md](#) or use this filled outline:

```
# ADR-NNNN: Short title

**Status:** Proposed
**Date:** YYYY-MM-DD
**Deciders:** OC CS Speckit project / your team

## Context
```

```
## Decision
## Consequences
### Positive
### Negative / tradeoffs
## Alternatives considered
## Related artifacts
```

Context — principles

- State the **problem or force** that required a choice (multi-user privacy, teachability, XAMPP, etc.).
- List **2–4 concrete questions** you had to answer.
- Mention constraints (stack, course, existing ADR-0001, ...).
- Do **not** jump to the solution here — that is Decision.

Decision — principles

- State what you chose in **one or two clear sentences**, then details (table or bullets).
- Be **specific**: technologies, boundaries, invariants ("server is source of truth", "cross-user → 404").
- Include **invariants** that every later feature must obey.
- Diagrams (text or Mermaid) are encouraged when they clarify trust boundaries or request flow.

Consequences — principles

- **Positive**: what you gain (testability, clarity, isolation).
- **Negative / tradeoffs**: what you give up (ops cost, no offline mode, ...). Honesty makes the ADR useful later.
- Call out follow-on work (e.g. "every feature must filter by `userId`").

Alternatives considered — principles

- Use a table: **Option | Why not**.
- Include real alternatives you discussed (SQLite, localStorage-only, MongoDB, `403` instead of `404`).
- "Why not" should be a reason, not "we didn't like it."

Related artifacts — principles

- Link **feature specs** that depend on this ADR.
- Link **Cursor rules** that enforce the decision day to day.
- Link **supersedes / superseded by** when status changes.
- Link **NFRs** or C4 diagrams when relevant.

Principles for writing a good ADR

1. **One decision, one file.** Split when two choices can change independently.
2. **Write for a future teammate.** Someone who was not in the room should understand *why*.
3. **Prefer durable rationale over temporary fashion.** "Works on developer laptops with XAMPP" ages better than "trendy this semester."
4. **Keep feature behavior out of ADRs.** Stories and Gherkin stay in `features/feature-N-*.md`.
5. **Keep coding patterns in rules.** Once Accepted, encode "how" in `.cursor/rules/` and point the ADR at those rules.
6. **Mark status honestly.** Use `Proposed` while debating; `Accepted` when the team commits; supersede instead of silently rewriting history.
7. **Link both ways.** ADR → `features/rules`; feature header `**Related:**` → ADR.
8. **Update when the decision changes.** Do not leave an Accepted ADR that contradicts the code — supersede or amend with a clear date/note.

9. **Stay concise.** A few pages max. Deep API tables belong in feature **API Requirements** or `features/reference/api.md` .
10. **Use this project's voice.** Deciders line can note OC CS Speckit / Todo example when the decision is kit-wide vs product-specific.

Workflow checklist

- ☐ Confirmed this is architecture/why — not a single feature story or a style rule alone
- ☐ Chose next free `NNNN` and kebab filename
- ☐ Filled Context, Decision, Consequences, Alternatives, Related
- ☐ Status set (`Proposed` or `Accepted`)
- ☐ Added row to [README.md](#) index
- ☐ Linked from the first feature that depends on it (`**Related:**`)
- ☐ If the decision implies ongoing coding constraints, added/updated a Cursor rule and linked it
- ☐ If an NFR Approach depends on this ADR, linked from [docs/nfr/quality-attributes.md](#)

Anti-patterns

Avoid	Do instead
ADR that only restates Feature 3 stories	Keep behavior in the feature spec; ADR for shared <i>why</i>
ADR with no alternatives	Always record what you rejected and why
Coding tutorial inside the ADR	Point at rules + short invariants
Changing an Accepted ADR silently after ship	New ADR or Superseded by ADR-XXXX
One giant "Architecture.md"	Numbered ADRs so decisions can be superseded one at a time

Relationship to feature writing

Need new product behavior?

→ features/writing-feature-requirements.md
features/writing-feature-design.md

Need a lasting architecture why?

→ docs/adr/writing-adrs.md (this file)

Need daily how-to for agents?

→ .cursor/rules/*.mdc

After an ADR is **Accepted**, feature **Data Ownership**, **API**, and security-related FRs should **align with** it — not reinvent isolation or auth in each file.